

Adaptive Priority Cycle Scheduling (APCS)

The APCS algorithm dynamically schedules traffic signals by considering:

- Vehicle Count
- Traffic Density
- Waiting Score

The algorithm generates a complete traffic signal cycle by ranking all directions according to a computed priority score. Green signal durations are allocated proportionally to the calculated priorities, while waiting scores ensure fairness across cycles

APCS introduces a cycle-based scheduling approach where a complete traffic signal cycle is generated using dynamically calculated priority scores. Unlike traditional fixed-time systems, APCS adapts signal ordering and green time allocation according to real-time traffic conditions while incorporating waiting-based fairness mechanisms.

Code and its Explanation:

```
directions = ["North", "South", "East", "West"]

waiting_counter = [0, 0, 0, 0]

MIN_GREEN = 10
MAX_GREEN = 40
CYCLE_NUMBER = 1
```

In this Snippet, it is indicating the **directions** described as(North, South, East and West).They might vary while being used in real world as(4 way intersection or 3 way intersection,etc). Naming the directions depends on the Traffic Signals location and the number of signals at that exact location are being used as a single cycle.

Next comes the **waiting_counter**, this is an initial store of waiting score like It stores a **waiting score**

MIN_GREEN, this acts as the minimum duration for the green signal time assigned to a particular direction as we are talking in **directions**

MAX_GREEN, this acts as the maximum duration that a direction can get as a green signal.

CYCLE_NUMBER, this is the initial cycle count of the whole signals duration in a junction.

```
def get_vehicle_counts():

    vehicles = []
```

```

print("\n=== Vehicle Input ===")

for direction in directions:
    count = int(
        input(f"Enter vehicles in {direction}: ")
    )
    vehicles.append(count)

return vehicles

```

The function **def get_vehicle_counts()**: acts as the function that works on getting the total vehicles that are scanned by the cameras installed. This function helps us to take input of the vehicles that are currently at the traffic signal and note the count of each direction and store it in **vehicles[]**.

The for loop in the function **for direction in directions**: this loop is taking the directions(North, South, East, West).

Inside the loop it is taking the input as count and the main part is **vehicles.append(count)**, Once we gave the input of the all the directions it is storing it in **vehicles[]** so that once we need them it will be easy for us to retrieve.

```

def calculate_density(vehicles):

    total = sum(vehicles)

    if total == 0:
        return [0] * len(vehicles)

    return [v / total for v in vehicles]

```

The function **def calculate_density(vehicles)**: uses the vehicles count that is stored in the function **def get_vehicle_counts()**:

The function then uses the formula:

$$Density_i = \frac{Vehicles_i}{\sum Vehicles}$$

Here **Density**, is being used by Vehicles(i) by sum(vehicles)

Vehicles(i) is known to be the vehicles count of a particular direction
sum(Vehicles) is known to be the total vehicle count that as in all directions.

The if statement that checks if total is 0 or not helps in not crashing the program if the total count is 0.

```
def calculate_priority(
    vehicles,
    densities,
    waiting_counter):

    priorities = []

    for i in range(len(directions)):

        vehicle_score = vehicles[i]

        density_score = densities[i] * 100

        waiting_score = waiting_counter[i]

        priority = (
            0.5 * vehicle_score +
            0.3 * density_score +
            0.2 * waiting_score
        )

        priorities.append(priority)

    return priorities
```

The function **calculate_priority()**, takes the values from vehicles, densities and waiting_counter.

The weights (**0.5, 0.3, and 0.2**) are currently chosen experimentally and can be adjusted based on traffic conditions or future optimization studies.

Formulas used:

DensityScore:

$$DensityScore_i = Density_i \times 100$$

PriorityScore:

$$Priority_i = 0.5(VehicleScore_i) + 0.3(DensityScore_i) + 0.2(WaitingScore_i)$$

GreenTime:

$$GreenTime_i = \frac{Priority_i}{\sum Priority} \times 60$$

WaitingScore:

$$WaitingScore_{new} = WaitingScore_{old} + CycleTime$$

DirectionSelection:

$$SelectedDirection = \arg \max(Priority_i)$$

Density Score is used to convert the density value into a percentage-like scale that can be easily used in the priority calculation.

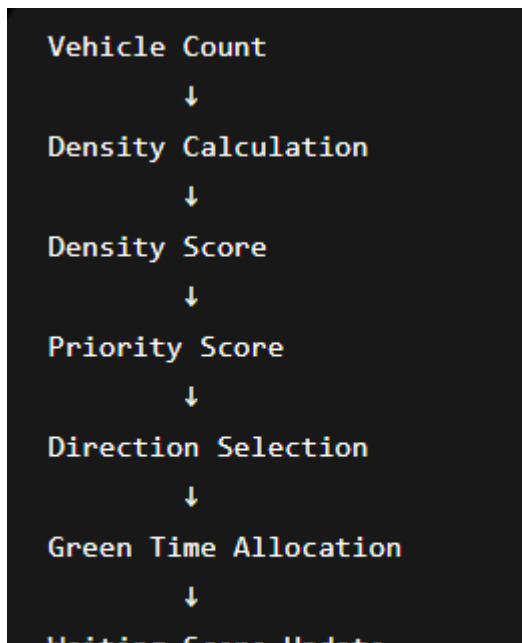
The density value is usually a decimal number between 0 and 1. Multiplying it by 100 scales the value, making it easier to combine with vehicle count and waiting score during priority calculation.

Green Time determines how long a direction should receive a green signal.

Waiting Score tracks how long a direction has been waiting without receiving a green signal.

Whenever a direction does not receive a green signal during a cycle, its waiting score increases. This gradually boosts its future priority and ensures that every direction eventually gets service.

Direction Selection determines the order of traffic signals within a cycle.



This is the flow of the current signal generation for each cycle.

```
def generate_cycle(priorities):  
  
    cycle = []  
  
    for i in range(len(directions)):  
        cycle.append(  
            (  
                directions[i],  
                priorities[i],  
                i  
            )  
        )  
  
    cycle.sort(  
        key=lambda x: x[1],  
        reverse=True  
    )  
  
    return cycle
```

The function **def generate_cycle(priorities)**: this works on giving the priorities for each direction and sort them in descending order as the table for the whole cycle.

Python's `list.sort()` and `sorted()` use **Timsort**.

Timsort is:

- Stable sorting algorithm
- Hybrid of Merge Sort and Insertion Sort
- Time Complexity:
 - Best Case: $O(n)$
 - Average Case: $O(n \log n)$
 - Worst Case: $O(n \log n)$

```
def allocate_green_times(priorities):

    total_priority = sum(priorities)

    green_times = []

    for priority in priorities:

        if total_priority == 0:
            green_time = MIN_GREEN

        else:
            green_time = (
                priority /
                total_priority
            ) * 60

        green_time = max(
            MIN_GREEN,
            min(MAX_GREEN, green_time)
        )

        green_times.append(green_time)

    return green_times
```

The function **def allocate_green_times(priorities):** takes the top order and then assigns the green time as the formula mentioned in page-3. Based on that the green signal duration for each direction will be assigned for the whole cycle.

```
def update_waiting_counter(cycle):

    served_index = cycle[0][2]

    for i in range(len(waiting_counter)):
```

```

if i == served_index:
    waiting_counter[i] = 0

else:
    waiting_counter[i] += 10

```

The function **def update_waiting_counter(cycle)** is responsible for updating the waiting scores of all directions after a signal cycle has been generated. When a direction receives a green signal, its waiting score is reset to 0 because it has been served.

For all other directions that did not receive immediate priority, the waiting score is increased by the predefined cycle time.

This mechanism ensures fairness in the traffic management system by preventing any direction from being ignored for an extended period.

As a direction waits through multiple cycles, its waiting score gradually increases, which contributes to a higher priority score in future cycles and improves its chances of receiving a green signal.

```

def print_cycle(
    cycle,
    green_times,
    priorities):

    print("\n==== COMPLETE SIGNAL CYCLE =====")

    for rank, item in enumerate(cycle, start=1):

        direction = item[0]
        index = item[2]

        print(
            f"{rank}. "
            f"{direction} | "
            f"Priority={priorities[index]:.2f} | "
            f"Green Time={green_times[index]:.2f}s"
        )

    print("\nSignal Order:")

    order = " -> ".join(
        [item[0] for item in cycle]
    )

```

```
)  
  
print(order)
```

The function **def print_cycle()**: then prints all the output that is needed for the traffic management system to work on printing the signal order, the time and the green time for each direction

```
def main():  
  
    global CYCLE_NUMBER  
  
    while True:  
  
        print(  
            f"\n\n===== CYCLE {CYCLE_NUMBER} ====="  
        )  
  
        vehicles = get_vehicle_counts()  
  
        densities = calculate_density(  
            vehicles  
        )  
  
        priorities = calculate_priority(  
            vehicles,  
            densities,  
            waiting_counter  
        )  
  
        cycle = generate_cycle(  
            priorities  
        )  
  
        green_times = allocate_green_times(  
            priorities  
        )  
  
        print("\n===== TRAFFIC REPORT =====")  
  
        for i in range(len(directions)):  
  
            print(  

```



```

        f"{directions[i]} | "
        f"Vehicles={vehicles[i]} | "
        f"Density={densities[i]:.2f} | "
        f"Waiting={waiting_counter[i]} | "
        f"Priority={priorities[i]:.2f}"
    )

    print_cycle(
        cycle,
        green_times,
        priorities
    )

    update_waiting_counter(
        cycle
    )

    CYCLE_NUMBER += 1

    choice = input(
        "\nGenerate next cycle? (y/n): "
    )

    if choice.lower() != "y":
        break

if __name__ == "__main__":
    main()

```

The function **def main()**: acts as the central controller of the traffic management system. It coordinates and executes all other functions in the correct sequence to perform a complete traffic signal cycle.

The function collects vehicle data, calculates traffic density, computes priority scores, generates the signal cycle, allocates green signal times, updates waiting counters, and displays the results.

By bringing together all individual modules, the **main()** function ensures that the entire traffic management algorithm operates as a single integrated system.

The flow chart and Pseudocode used in the **def main()**:

```
Start
Input vehicle counts
Calculate density
Calculate priority scores
Generate cycle order
Allocate green times
Display cycle
Update waiting scores
Repeat
End
```

```
Get Vehicle Counts
↓
Calculate Density
↓
Calculate Priority Scores
↓
Generate Signal Cycle
↓
Allocate Green Times
↓
Display Traffic Report
↓
Update Waiting Counters
↓
Generate Next Cycle
```

Why APCS is Considered an Algorithm

The Adaptive Priority Cycle Scheduling (APCS) system is considered an algorithm because it follows a well-defined sequence of computational steps to solve a specific problem, namely traffic signal scheduling and traffic flow optimization at an intersection.

The algorithm takes traffic-related inputs such as vehicle counts, traffic density, and waiting scores from multiple directions. These inputs are processed through mathematical formulas to calculate priority scores for each direction. Based on the calculated priorities, the algorithm generates a complete signal cycle, allocates green signal durations, updates waiting scores, and repeats the process for subsequent cycles.

Unlike a simple traffic timer that operates with fixed signal durations, APCS makes dynamic decisions based on real-time traffic conditions. The algorithm continuously evaluates traffic conditions and adjusts signal priorities accordingly, making it adaptive in nature.

The current version of APCS focuses on three primary factors:

1. Vehicle Count
2. Traffic Density
3. Waiting Score

These factors are combined to generate a priority score that determines signal order and green signal allocation.

Current Limitations and Future Enhancements

Although APCS successfully demonstrates adaptive traffic scheduling, several advanced factors have not yet been incorporated into the current version of the algorithm.

The following enhancements are planned for future versions:

1. **Traffic Growth Rate Analysis**
 - The current algorithm evaluates only the present vehicle count.
 - Future versions will track how quickly traffic is increasing in each direction.
 - This will allow the system to identify emerging congestion before it becomes severe.
2. **Cycle Stability Mechanism**
 - The current algorithm can significantly change signal ordering between cycles if traffic conditions fluctuate.
 - Future versions will introduce a cycle stability factor to reduce unnecessary signal reordering while maintaining adaptability.
3. **Traffic Prediction Module**
 - The current system reacts to existing traffic conditions.
 - Future versions will predict near-future traffic conditions using historical and real-time data, enabling proactive traffic management.
4. **Emergency Vehicle Prioritization**
 - The current algorithm treats all vehicles equally.

- Future versions will detect and prioritize emergency vehicles such as ambulances, fire trucks, and police vehicles.

5. Adaptive Weight Optimization

- The current priority weights are manually assigned.
- Future versions may use optimization techniques or machine learning models to automatically determine the most effective weights for different traffic environments.

These enhancements represent the future development roadmap of APCS and are expected to improve traffic efficiency, responsiveness, fairness, and real-world applicability.